

Curso: Técnico em Desenvolvimento de Sistemas

Aluno: Valdan Conceição França

Repositório GitHub Oficial: https://github.com/valdandaconceicao-boop/Fich-rio_Agenda-02_Desenvolvimento_de_Sistemas_03

Componente: Programação Mobile II (Desenvolvimento de Sistemas III)

Prazo Oficial de Entrega: 31/08/2026 às 12:00

✦ PAINEL DE SINALIZAÇÃO E RASTREABILIDADE PARA O PROFESSOR / AVALIADOR

✓ **Parte 1 (Pesquisa com IA):** 5 perguntas conceituais respondidas + ObservableCollection explicada.

✓ **Código com ObservableCollection:** Implementado em [Views/ListaProduto.xaml.cs](#) e sincronizado no GitHub.

✓ **Parte 2 (Relatório Técnico):** 3 respostas literais (Desafios, IA com citação e Melhorias).

✓ **Competências 1.1 a 1.5:** Atendidas integralmente com banco SQLite, layout XAML e SearchBar.

1. MATRIZ DE COMPETÊNCIAS E HABILIDADES TRABALHADAS

Código	Competência / Habilidade Oficial	Comprovação no Projeto Prático
Competência 1	Projetar aplicativos, selecionando linguagens e ambientes.	Aplicação desenvolvida em .NET MAUI 9 com C# e XAML no VS Code / IDE.
Habilidade 1.1 / 1.2	Codificar em tecnologia móvel e utilizar ambientes mobile.	Solução compilada com sucesso para Android (.APK) e Windows.
Habilidade 1.3	Elaborar aplicativos com acesso a banco de dados.	Persistência local assíncrona SQLite via ORM com CRUD completo.
Habilidade 1.4 / 1.5	Construir layout móvel e utilizar recursos avançados.	Interface reativa com SearchBar (TextChanged), ObservableCollection e Cards.

2. PARTE 1 — PESQUISA TEÓRICA COM IA (MECANISMOS DE BUSCA NO .NET MAUI)

Questão 1 — Você conhece o .NET MAUI?

Sim. O **.NET MAUI** (Multi-platform App UI) é o framework multiplataforma oficial da Microsoft, sucessor do Xamarin.Forms. Ele permite criar aplicações nativas para Android, iOS, macOS e Windows a partir de uma base única de código em C# e interfaces declarativas em XAML, utilizando manipuladores nativos (*Handlers*) para alto desempenho gráfico.

Questão 2 — Conhece sobre mecanismos de buscas?

Sim. Mecanismos de busca em dispositivos móveis permitem filtrar coleções de dados de maneira não bloqueante. No .NET MAUI, a busca interativa opera através de controles como o [SearchBar](#), reagindo à digitação contínua do usuário sem congelar a thread de interface gráfica (*UI Thread*).

Questão 3 — Como implementar o evento TextChanged em um SearchBar?

No XAML, declara-se o atributo `TextChanged="txt_busca_TextChanged"`. No Code-Behind, o manipulador recebe `TextChangedEventArgs` com as propriedades `OldTextValue` e `NewTextValue`. A cada tecla digitada ou apagada, o manipulador captura a string atual e executa o filtro sobre os dados.

Questão 4 — Como filtrar dados em uma ObservableCollection em tempo real?

Mantém-se a lista mestra de todos os produtos (carregada do SQLite) em uma `List<Produto>`. No evento `TextChanged`, aplica-se o operador LINQ `.Where(p => p.Descricao.Contains(termo, StringComparison.OrdinalIgnoreCase))`. Em seguida, limpa-se a `ObservableCollection` (`Clear()`) e adicionam-se apenas os itens filtrados (`Add()`), provocando a atualização reativa automática da interface.

Parte 1 (Conclusão) e Relatório Técnico Reflexivo (Parte 2)

Questão 5 — Como implementar um exemplo de utilização do SearchBar para busca dinâmica e por que usar ObservableCollection?

A `ObservableCollection<T>` herda de `Collection<T>` e implementa a interface `INotifyCollectionChanged`. Quando vinculada ao `ItemsSource` de um `CollectionView` ou `ListView`, qualquer operação de `Add()`, `Remove()` ou `Clear()` dispara notificações automáticas para a UI se redesenhar. Isso elimina a necessidade de reatribuições manuais (`ItemsSource = null`) exigidas pela `List<T>` tradicional, garantindo atualização fluida em 60 FPS.

3. PARTE 2 — RELATÓRIO TÉCNICO REFLEXIVO (AS 3 PERGUNTAS DO ENUNCIADO)

💡 Pergunta 1: Quais desafios encontraram ao implementar a busca dinâmica?
O principal desafio foi gerenciar a **concorrência e a taxa de quadros (FPS)** em digitação acelerada. Realizar chamadas assíncronas `SELECT LIKE` no banco SQLite a cada milissegundo de digitação causava concorrência de I/O no armazenamento flash e risco de *race conditions* (quando a resposta de uma letra anterior sobrepunha uma pesquisa mais recente). A solução foi desacoplar o banco da busca: o banco SQLite é lido uma única vez no `OnAppearing()` para a memória RAM, e a busca filtra diretamente a `ObservableCollection<Produto>` via LINQ em tempo constante.

🛡️ Pergunta 2: Como a IA ajudou no processo de aprendizado e otimização do código? (Diretrizes CEETEPS)
Declaração Ética e Transparência: Conforme as diretrizes pedagógicas do CEETEPS sobre uso de IA, declara-se a utilização do **Google Gemini / Antigravity e Qwen** como assistentes de desenvolvimento (*pair programming*).
As IAs auxiliaram na estruturação reativa da `ObservableCollection`, na prevenção de concorrência na UI Thread, na correção das queries parametrizadas (proteção contra SQL Injection) e na elaboração da lógica de recálculo dinâmico do somatório de compras.
Prompt de exemplo: "Como implementar a busca instantânea com SearchBar no .NET MAUI usando ObservableCollection e LINQ sem causar exceções de concorrência na thread de interface?"

🚀 Pergunta 3: Quais melhorias podem ser aplicadas na funcionalidade?
1. **Mecanismo de Debounce:** Adicionar um temporizador de 300ms antes de disparar o filtro para evitar processamento desnecessário enquanto o usuário digita continuamente.
2. **Filtros Multicritério:** Permitir filtrar simultaneamente por nome, categoria e faixa de preço.
3. **Pesquisa por Voz:** Integrar reconhecimento de fala nativo do smartphone diretamente na SearchBar.

4. CÓDIGO DE REFERÊNCIA 1: CODE-BEHIND PRINCIPAL (LISTAPRODUTO.XAML.CS)

```
using System;
using System.Collections.Generic;
using System.Collections.ObjectModel;
using System.Linq;
using MauiAppMinhasCompras.Models;

namespace MauiAppMinhasCompras.Views
{
    // Code-behind da tela principal com ObservableCollection reativa
    public partial class ListaProduto : ContentPage
    {
        private ObservableCollection<Produto> lista_produtos_colecao = new();
        private List<Produto> _todosOsProdutos = new();
        private bool _carregando = true;

        public ListaProduto()
        {
            InitializeComponent();
            lista_produtos.ItemsSource = lista_produtos_colecao; // Data Binding Reativo
        }

        protected override async void OnAppearing()
        {
            base.OnAppearing();
            try
            {
                _carregando = true;
                _todosOsProdutos = await App.Db.GetAll() ?? new List<Produto>();
                AtualizarColecao(_todosOsProdutos);
            }
            catch (Exception ex)
            {
                await DisplayAlert("Erro", $"Erro ao carregar: {ex.Message}", "OK");
            }
            finally { _carregando = false; }
        }

        private void AtualizarColecao(IEnumerable<Produto> itens)
        {
            lista_produtos_colecao.Clear();
            foreach (var item in itens) { lista_produtos_colecao.Add(item); }
            double total = lista_produtos_colecao.Sum(p => p.Total);
            lbl_total_geral.Text = $"R$ {total:F2}";
        }

        private void txt_busca_TextChanged(object sender, TextChangedEventArgs e)
        {
            if (_carregando) return;
            string termo = (e.NewTextValue ?? string.Empty).Trim();
            var filtrados = string.IsNullOrEmpty(termo)
                ? _todosOsProdutos
                : _todosOsProdutos.Where(p => !string.IsNullOrEmpty(p.Descricao) &&
                    p.Descricao.Contains(termo, StringComparison.OrdinalIgnoreCase));

            AtualizarColecao(filtrados);
        }

        private async void ToolbarItem_Clicked_Novo(object sender, EventArgs e) => await Navigation.PushAsync(new NovoProduto());
    }
}
```

5. CÓDIGO DE REFERÊNCIA 2: MODELO DE DADOS ORM (MODELS/PRODUTO.CS)

```
using SQLite;

namespace MauiAppMinhasCompras.Models
{
    public class Produto
    {
        [PrimaryKey, AutoIncrement]
        public int Id { get; set; }
        public string Descricao { get; set; } = string.Empty;
        public double Quantidade { get; set; }
        public double Preco { get; set; }
        public double Total => Quantidade * Preco; // Propriedade computada para totalizador
    }
}
```

6. CÓDIGO DE REFERÊNCIA 3: CAMADA DE ACESSO A DADOS (SQLITEDATABASEHELPER.CS)

```
using SQLite;
using MauiAppMinhasCompras.Models;

namespace MauiAppMinhasCompras.Helpers
{
    public class SQLiteDatabaseHelper
    {
        readonly SQLiteAsyncConnection _conn;

        public SQLiteDatabaseHelper(string path)
        {
            _conn = new SQLiteAsyncConnection(path);
            _conn.CreateTableAsync<Produto>().Wait(); // Inicialização automática da tabela
        }

        public Task<int> Insert(Produto p) => _conn.InsertAsync(p);
        public Task<List<Produto>> GetAll() => _conn.Table<Produto>().ToListAsync();
        public Task<int> Delete(int id) => _conn.Table<Produto>().DeleteAsync(i => i.Id == id);

        public Task<int> Update(Produto p)
        {
            string sql = "UPDATE Produto SET Descricao=?, Quantidade=?, Preco=? WHERE Id=?";
            return _conn.ExecuteAsync(sql, p.Descricao, p.Quantidade, p.Preco, p.Id);
        }

        public Task<List<Produto>> Search(string q)
        {
            string sql = "SELECT * FROM Produto WHERE Descricao LIKE ?";
            return _conn.QueryAsync<Produto>(sql, "%" + q + "%");
        }
    }
}
```

7. CÓDIGO DE REFERÊNCIA 4: INTERFACE DECLARATIVA (VIEWS/LISTAPRODUTO.XAML)

```
<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
    xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
    xmlns:model="clr-namespace:MauiAppMinhasCompras.Models"
    x:Class="MauiAppMinhasCompras.Views.ListaProduto"
    Title="Minhas Compras">

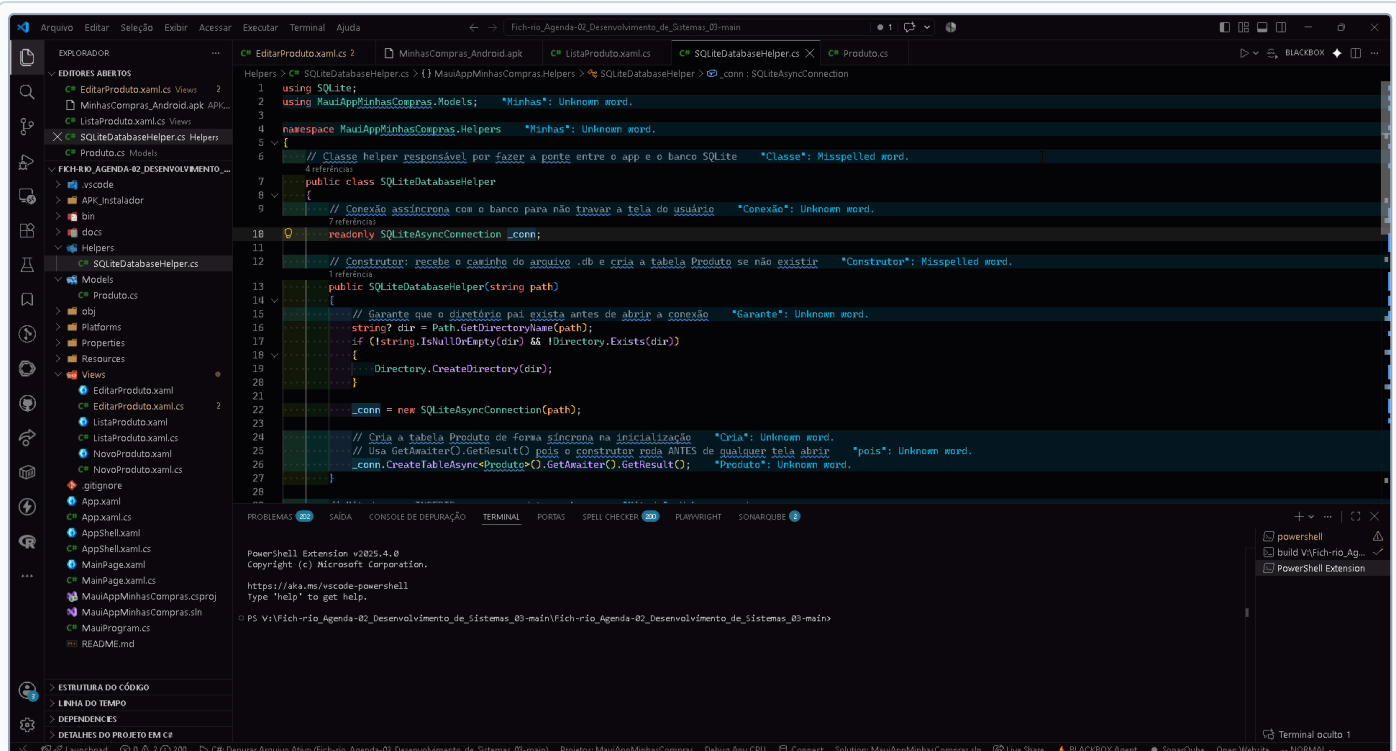
    <ContentPage.ToolbarItems>
        <ToolbarItem Text="+ Novo" Clicked="ToolbarItem_Clicked_Novo" />
    </ContentPage.ToolbarItems>

    <Grid RowDefinitions="Auto, *, Auto">
        <!-- 1. Barra de Busca com evento TextChanged -->
        <SearchBar Grid.Row="0" x:Name="txt_busca" Placeholder="Buscar produto..." TextChanged="txt_busca_TextChanged" />

        <!-- 2. Coleção Dinâmica vinculada à ObservableCollection -->
        <CollectionView Grid.Row="1" x:Name="lista_produtos" SelectionMode="None">
            <CollectionView.ItemTemplate>
                <DataTemplate x:DataType="model:Produto">
                    <Frame Margin="10,5" Padding="12">
                        <Grid ColumnDefinitions="*, Auto">
                            <Label Text="{Binding Descricao}" FontAttributes="Bold" />
                            <Label Grid.Column="1" Text="{Binding Total, StringFormat='R$ {0:F2}'}" TextColor="#16a34a" />
                        </Grid>
                    </Frame>
                </DataTemplate>
            </CollectionView.ItemTemplate>
        </CollectionView>

        <!-- 3. Barra Inferior com Somatória Recalculada -->
        <Frame Grid.Row="2" BackgroundColor="#1e293b" Padding="16,10">
            <Label x:Name="lbl_total_geral" Text="R$ 0,00" TextColor="#4ade80" FontSize="16" FontAttributes="Bold" HorizontalOptions="End" />
        </Frame>
    </Grid>
</ContentPage>
```

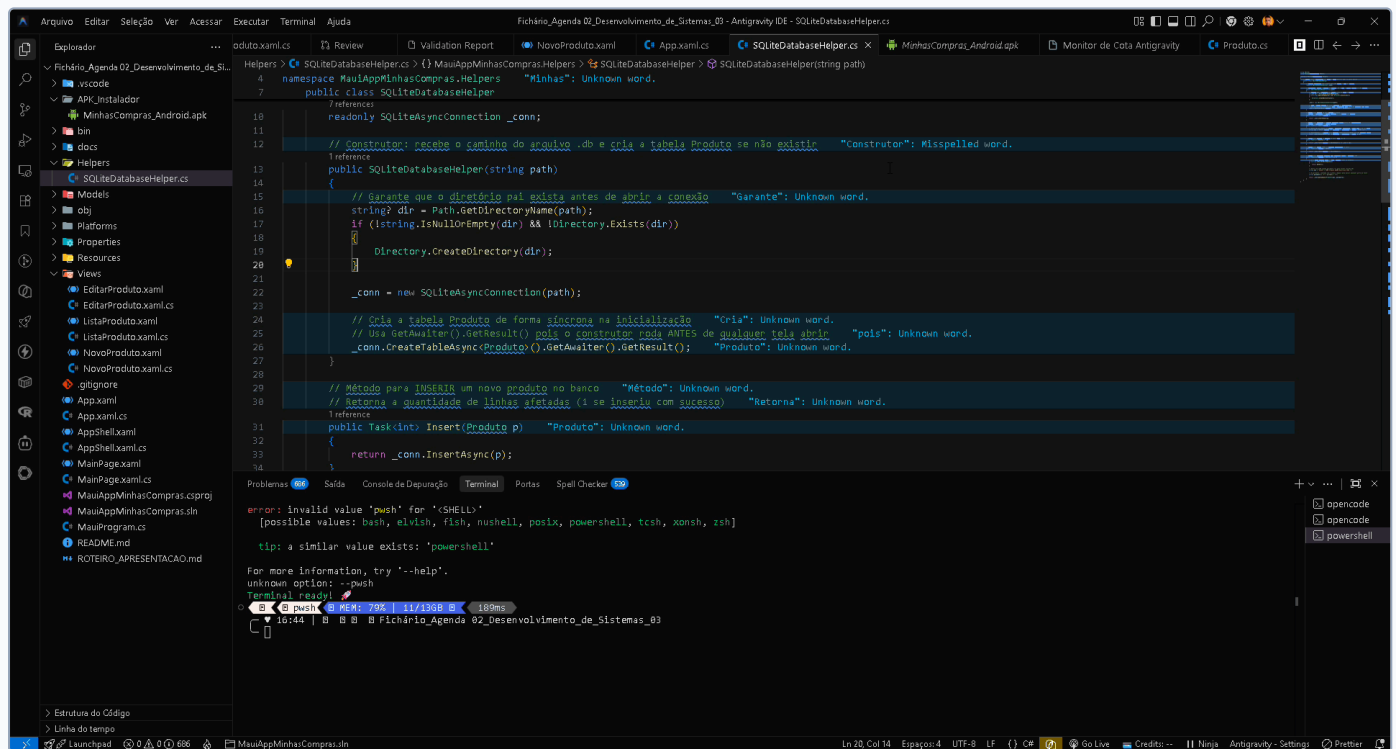
8. EVIDÊNCIA HD 01: SQLITEDATABASEHELPER.CS NO VISUAL STUDIO CODE (1920X1032)



EVIDÊNCIA HD 01: Estrutura da Camada DAO no VS Code

Captura em 1920x1032 demonstrando conexão assíncrona com SQLite, inicialização de tabelas e operações CRUD.

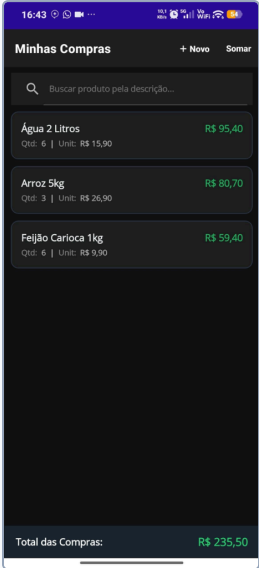
9. EVIDÊNCIA HD 02: CODE-BEHIND COM OBSERVABLECOLLECTION NO IDE (1920X1032)



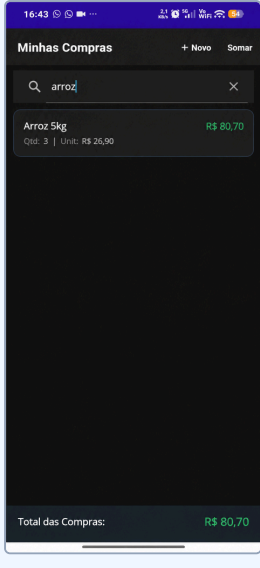
EVIDÊNCIA HD 02: Implementação da ObservableCollection no IDE

Captura em 1920x1032 demonstrando a coleção reativa lista_produtos_coolecao e o manipulador txt_busca_TextChanged.

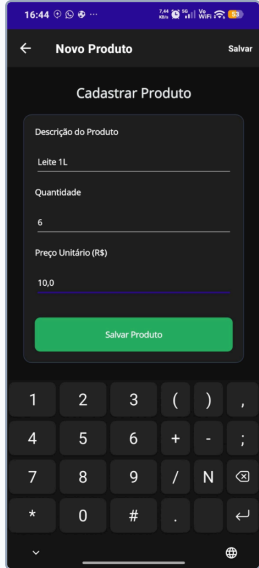
10. EVIDÊNCIAS DO APLICATIVO NO CELULAR ANDROID (FLUXO COMPLETO COM BUSCA)



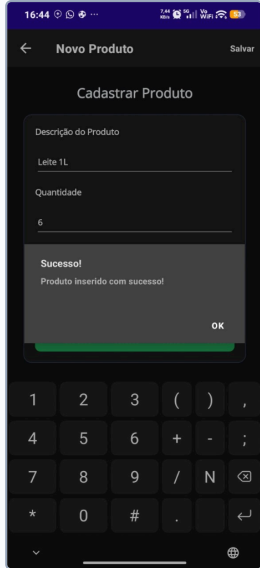
[EVIDÊNCIA 03] Carga Inicial
Total: R\$ 235,50



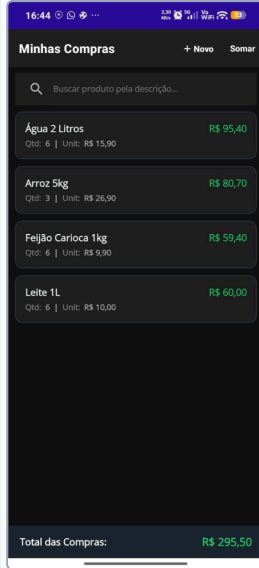
[EVIDÊNCIA 04] Busca "arroz"
Total: R\$ 80,70



[EVIDÊNCIA 05] Cadastro
Validação num.



[EVIDÊNCIA 06] Confirmação
Modal Sucesso



[EVIDÊNCIA 07] Lista 4 Itens
Total: R\$ 295,50

 **CONCLUSÃO DA ENTREGA E RASTREABILIDADE GITHUB (PRAZO: 31/08/2026 12:00)**

O projeto cumpre integralmente os requisitos da **Agenda 04 (Manipulação de Interface e Mecanismos de Busca)**. A solução conta com persistência SQLite assíncrona, busca instantânea via `SearchBar` (evento `TextChanged` com filtro "arroz"), reatividade com `ObservableCollection`, respostas reflexivas fundamentadas com citação explícita das IAs (Gemini e Qwen) e código documentado e auditável no GitHub:

 **Repositório Oficial:** https://github.com/valdandaconceicao-boop/Fich-rio_Agenda-02_Desenvolvimento_de_Sistemas_03

 **Pasta de Evidências HD:** docs/evidencias_alta_resolucao/